

**Automata Explorer
User Guide
Draft 01**

Introduction

Automata explorer (AE) is an application that is designed for a user to perform all the tasks and create all the files and conditions required to be able to run and display a computative iterative process to represent a cellular automata process represented in the form of a hexagonal grid.

In this cellular automata application, the user creates the cellular automata rule conditions in the form of a series of c/c++ like conditional statements that are then used as a basis to modify the values stored in a hexagonal grid structure to emulate a cellular automata iteration process that progresses from one state of the hexagonal grid to the next.

The desire for this application originates from a previous unfinished project called Virtual Worlds where a very crude and unsatisfactory method and attempt to create a kind of cellular automata process on a 3D hexagonal grid of close packing spheres of equal radius that can be represented as a 3D grid of Trapezo-rhombic dodecahedral voxels. Though it worked in the test conditions, it was not easy to implement or a desirable solution. The goal here is to eventually come up with a robust and complete cellular automata solution that can be implemented on any specified 2D/3D grid structure in either the traditional, or desired hexagonal format. This is the first iteration of using the code for a 2D hexagonal grid derived from the Virtual Worlds project into a complete and easy to use solution that will be expanded into a 3D solution. This is why the current executable file of this project is called VWHA.exe which stands for Virtual Worlds hexagonal automata.

What will be described in this user manual are the methods and procedures to create and implement a cellular automata process on a 2D hexagonal grid structure and visualise it in a form that so it can be examined and tested.

The cellular automata application as it is currently implemented only uses integer values for the values that each cellular automata grid cell can have. It is intended that a future version of this application will be able to utilise floating point or real numbers and perhaps even boolean.

Application Examples File Structure

The cellular explorer application utilises a method of exporting and importing ASCII text files to save and load the various data elements that the application can use to define and execute a cellular automata process. There are example files that exist within the same directory that the binary executable (VWHA.exe) exists. The binary executable resides in the bin/x64/Release directory as do these example files. The file structure of this directory is

Brushes	(files of brushes to "paint" automata grid values)
Fonts	(application fonts)
Gradients	(files of color gradients to display automata grid values)
Grids	(files of 2D hexagonal grids)
Icons	(application icons)
Rules	(files of example automata rules to perform automata iterations)

where within the Rules directory there are sub directories called

Automata	(files of complete automata iteration result rules)
Rules	(files of automata rule definitions)
SubRules	(files of subrules that can be imported in any rule definition)

This user guide will use some of these example files to demonstrate a functionality or shortcut a process to be performed, and act as a basis for the user to save and create their own cellular automata projects. It is essential that the Fonts and Icons directories and files exist in the same directory as the application executable or the application will not start, and if it does, it will potentially crash.

Potential Application Crash

The cellular explorer application while in development had experienced a spate of crashes that occurred at unexpected and random times for seemingly no reason at all. However after extensive refactoring of code and finding where the problems were originating from, these crashes seemed to be eliminated except when selecting the file dialogue to save and load the various application files. This file dialogue was a dependency and it seemed that either visual studio compilation and/or windows 11 was incompatible with its code. This seems to have been resolved however by downloading and installing the latest version of this dependency called tinyFileDialog. As of present no more crashes have occurred thus far in many test runs, but if it does on the users machine, it seems to be a windows 11 related problem with the tinyFileDialog having clashes or memory problems with opening at its last opened directory. If the user is finding this problem on their machine, restarting the application and trying to continue from where the application crashed, the user may find it does not crash. So it is wise to save any work done regularly and in small steps if this problem is occurring.

Quick Start

- 1 : On starting the application the user is presented with a blank screen with a menu list under the Project banner (Fig 01). Left click on this menu option and select the menu option New Automata Project. This will open up a dialogue box to specify the dimensions and an initial value for the hex grid that the cellular automata is to be performed within (Fig 02). Accept the default values and select the Create Global Hex Grid button with the left mouse button to generate a default hex grid. A series of ImGui widgets should then be displayed in the screen as illustrated as in Fig 03.

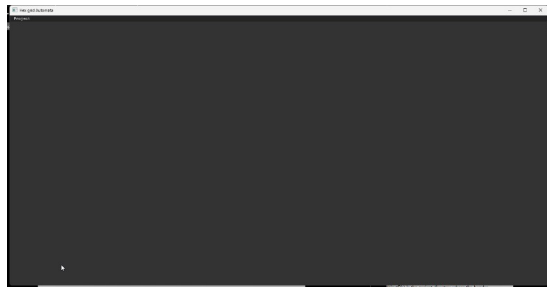


Fig 01

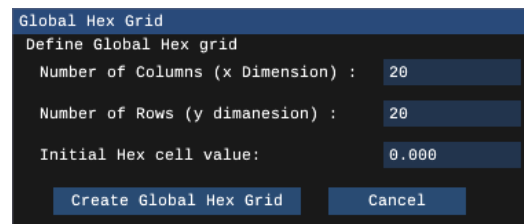


Fig02

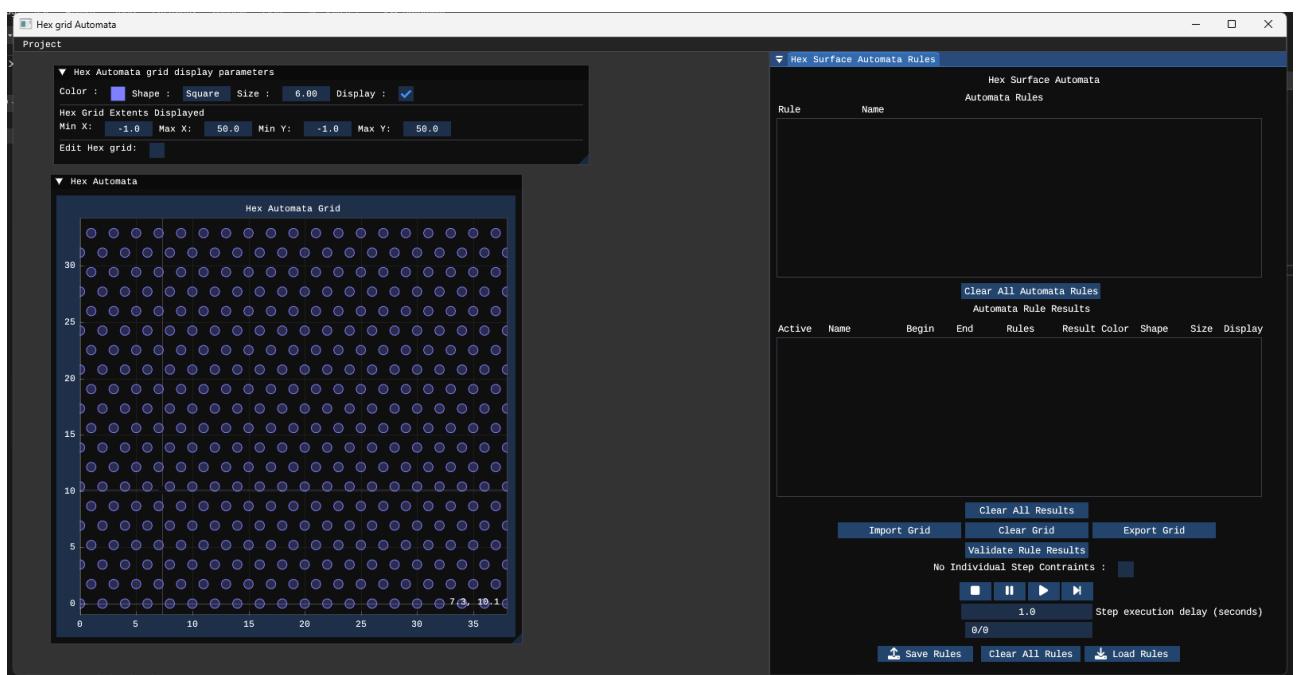


Fig 03

- 2 : Under the Hex Surface Automata Rules widget is a button Load Rules. Select it and a file dialogue widget should be displayed to select a file of automata rules should appear and most likely will default to the directory that the application was started in. Navigate to the directory called Rules/Automata. A selection of example rules should appear. Select the rules file called conroy_game_of_life_HGAR.txt to load this set of rules.
- 3 : Before the rules for this selection can be applied, the rules need to be validated. Select the Validate Rules Results button. A dialogue to indicate the rules have been validated should appear. Select the OK button to close the dialogue.
- 4 : Under the Hex Automata grid display parameters widget is a checkbox with the label Edit Hex Grid:(**Fig 04**). Select it to have a tick mark displayed in the check box, a widget Edit Hex Automata Grid should appear.(**Fig 05**).

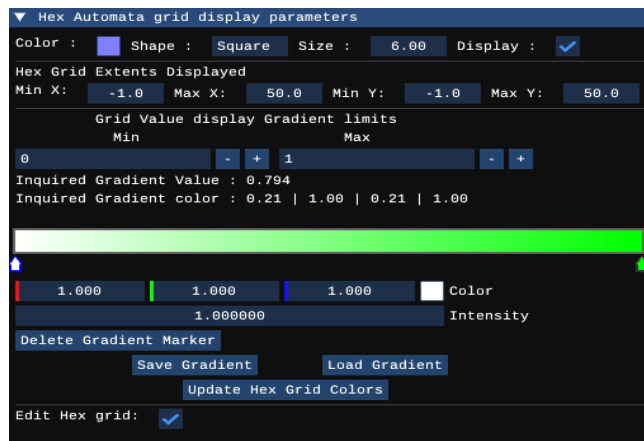


Fig 04

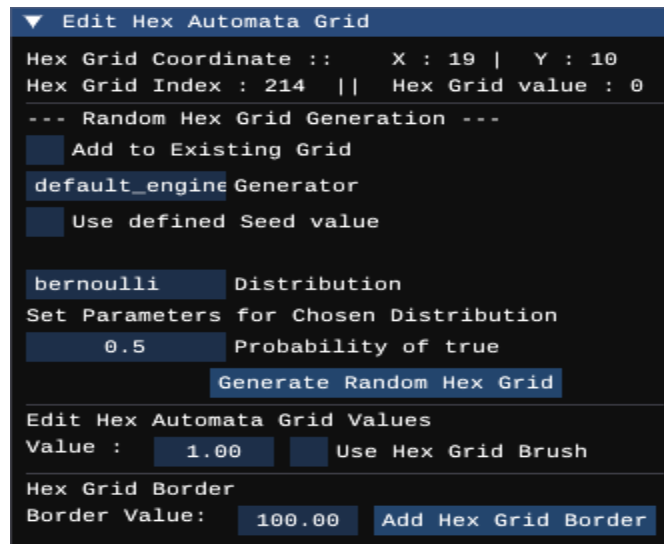


Fig 05

- 5 : Within the Edit Hex Automata Grid widget is a section called Random Hex Grid Generation. Select the drop down combo menu next to the label Distribution and select bernoulli. The display under Set Parameters for Chosen Distribution should change to give an entry of 0.5 next to a text label of Probability of true. Accept this value and select the button directly under this with the label Generate Random Hex Grid. A random distribution of grid values should now be displayed in the plotted hex grid.

- 6 : Now to perform the cellular automata task upon this defined grid, in the Hex Surface Automata Rules widget are a series of control buttons under the validate the rules results button that look similar to that of a video or audio player. In order left to right are stop, pause, play, and next step. Press the far right button that looks like an arrow head and vertical line to process the first automata step for this grid and defined rule set. The grid display will change to indicate what rules have been applied corresponding to the list of Automata rule results. By selecting the display check box at the end of each rule result the user can turn on and off the rule result display data. This display is useful for debugging automata rules and results. Deselecting all the displays the user can see the progress of the automata rules applied to the hex grid as it goes through each iteration step. Repeat pressing the next step control button until it stops at step 10, or the automata grid does not change or becomes blank. Upon reaching step 10, the user can reset the automata step process by clicking on the far left control button that looks like a square or stop button. This will reset the automata step back to zero and the user can again press the next step button to progress the cellula automata grid to the next iteration or state.(Fig 06)

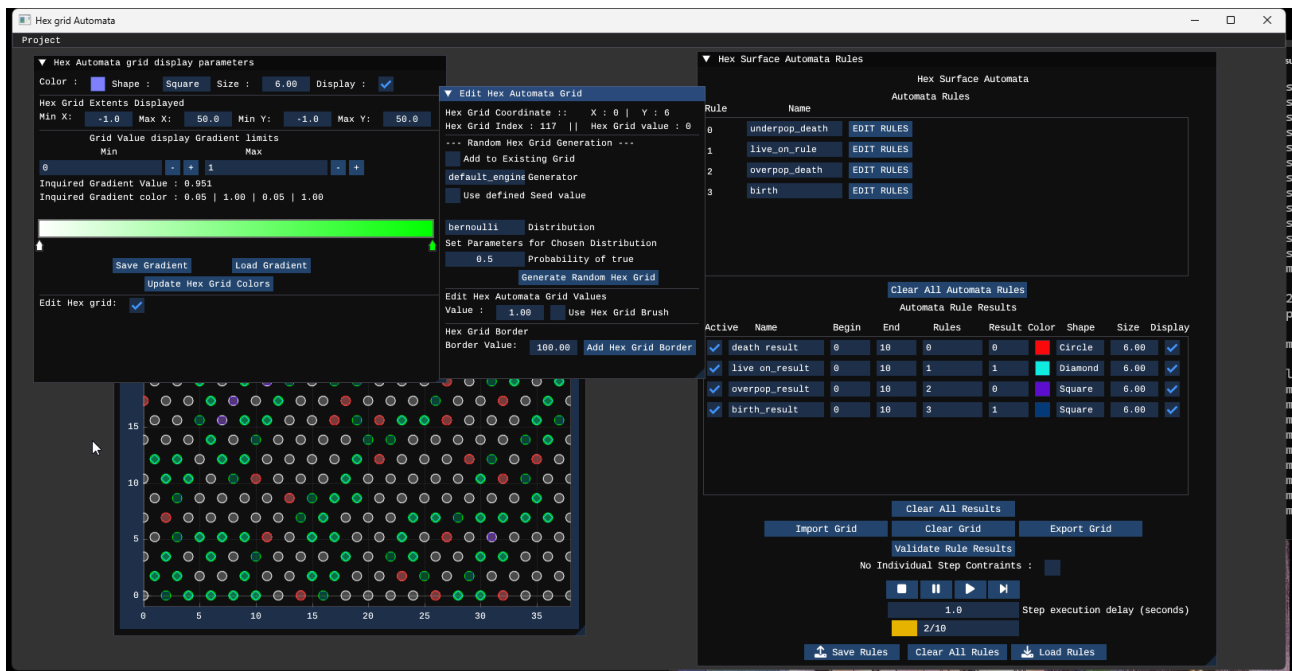


Fig 06

This is the most basic of functionality that is explained for this application. Defining rules and initiating a grid to use those rules will be described in a future user guide publication.

1 : Interface

The automata explorer application is essentially split into four partitions for the user to interact with.

- 1.1 : **The main window.** This is where the user creates and exits the application, and where other project or application specifics such as preferences are to be found and interacted with by selecting menu options and pop up widgets or dialogues. Currently this is limited to the user creating a new 2D hexagonal grid automata and exiting the application. In future more should be added as required.
- 1.2 : **The hexagonal grid preferences and editor window widget.** This is where the user specifies the display parameters of the hexagonal grid that is displayed, and to interact with various widgets to edit and examine the values of the hexagonal grid to be used in the cellular automata step process.
- 1.3 : **The hexagonal grid Automata rules window widget.** This is where the user creates, loads and modifies the cellular automata rules. It is also the controls for the execution of the cellular automata rules exist and where the user can import or export a cellular automata grid from and to disk storage.
- 1.4 : **The interactive hexagonal grid display.** This is a widget window that displays the current cellular automata state in a graphical form that the user can interact with to edit the current cellular automata state, or with the user of the mouse cursor and graphical display, enquire and examine the hexagonal grid data values of the current iterative state

The automata explorer application window has docking capabilities that give the user the ability to arrange and dock these window widgets in any arrangement the user desires. All the application user interface window widgets are displayed in action in **Fig 06**.

2: Application User Interface

2.1 : Creating a Cellular Automata project and defining the hexagonal grid to be used.

In step one of the section Quick Start, it was explained how to create a new cellular automata project from the main application window (**Fig 01**) by selecting the main window menu option Project→New Automata Project to open a dialogue window as given in **Fig 02**.

This project dialogue window has options to specify the dimensions of the hexagonal grid that is to be used to perform the cellular automata iteration procedure. The default is a regular 20 by 20 2D hexagonal grid but if the user wants a different sized hexagonal grid, the values are entered here to define the desired hexagonal grid size.

(Note that this application has not been tested memory wise for grid sizes or dimensions greater than 1000 by 1000. Also note that the processing times may slow considerably as the size increases as well and thus far no testing of processing grids above 100 by 100 have been performed to gauge processing performance.)

Before creating the hexagonal grid, the user can specify an initial base value that every hexagonal grid cell has assigned to it. Currently, the hexagonal grid can only be assigned an integer value. Therefore the only cellular automata iteration processing is that involving integer values. So any cellular automata models and rules that are to be created can only be those that use integer values. The default initial integer value for every hexagonal grid cell is zero, and if the user wants to have a different initial value for every hexagonal grid cell, this entry just needs to be changed to that value.

If the user wishes to not create a hexagonal grid and have control restored to the main application window, to perform actions from there, the cancel button can be selected to do so.

Once all the desired inputs are entered to create the hexagonal grid for the cellular automata iteration project, the user selects the Create Global hex Grid button. This will create an initial hexagonal grid and display it in a window widget alongside with two additional widget windows as illustrated in **Fig 03** in the Quick Start section.

2.2 : Defining and editing the Cellular Automata hexagonal grid to be used.

After performing the task of creating the hexagonal grid and opening up the cellular automata widgets as described in section 2.1 all of the main project window widgets should be displayed similar to that given in **Fig 03** of the Quick Start section. One of those window panels is the hexagonal grid display similar **Fig 07**.

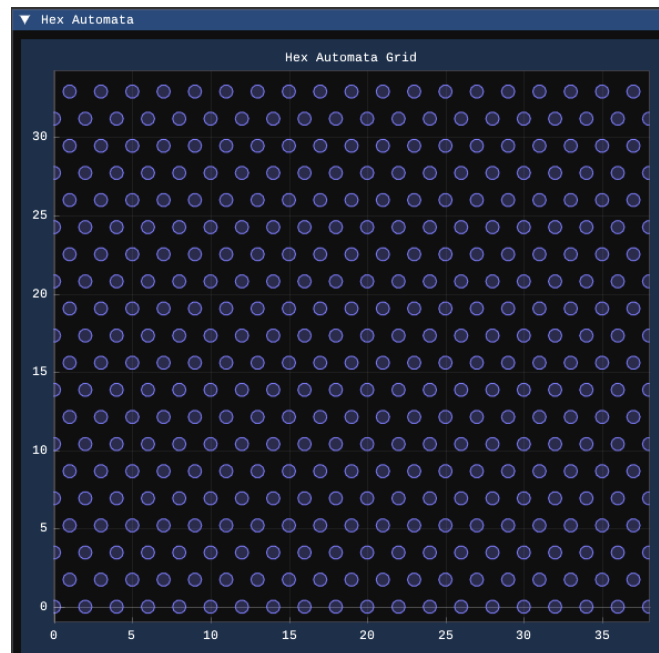


Fig 07

This display is initially that of the created hexagonal grid with the defined initial value at each hexagonal grid cell. This display is interactive in that the user can change its size the same way any GUI window can be changed. Upon changing the size of any side, the width or height of the window will change to keep the display at a constant width-height ratio. To perform other functions, the user needs to click on or in the display window to make it the current active window. The user can then perform other functions such as to zoom in or out of the hexagonal grid display.

To zoom in or out, the user uses the scroll button on their mouse. There will be a minimum and maximum zoom in and zoom out limitation.

The user can also pan the display by pressing the left mouse button and then move the mouse. The limits on how much the user can pan is dictated by the display constraints.

As the user interacts with this window widget, there is displayed a cross hairs cursor indicating the position of a cursor of interaction with the grid display. In the bottom right hand corner is a display in Cartesian coordinates, the current cursor location within the window widget. Also there are displayed reference x-y axis Cartesian coordinates along the left and bottom of the display.

To perform a hexagonal automata iteration process, the hexagonal grid needs to be defined with an initial state of values and a set of cellular automata rules created or loaded and the rules initialise or defined for use. The order in which this is done does not matter, and as such it does not matter in which order the following sections 2.3 and 2.4 are performed.

2.3 : Defining and editing the Cellular Automata hexagonal grid to be used.

After performing the task of creating the hexagonal grid and opening up the cellular automata widgets as described in section 2.1, the user can define the display parameters and edit the initial hex grid as required to be processed by the defined automata rules that the user has created or loaded into the application project. This is done through the window widget with the label “Edit Hex Automata Grid” as displayed in **Fig 05** in the Quick Start section. What is done in this widget panel will effect the display and data values of the hexagonal grid window widget. When this selection is activated, User interaction with the hexagonal grid display is changed. Any left mouse clicks in the hexagonal grid display will be to edit the current hexagonal grid cell that the mouse cursor is hovering over. To zoom in and out and pan, the user needs to press the shift key before using the mouse scroll or left mouse button.

2.3.1 Hexagonal grid cell display

The top section of this widget are the display parameters that define the displayed hexagonal grid in the hexagonal grid display window widget. The user can change the color, grid cell shape and size of that shape in the display of the hexagonal grid. An option to display the hexagonal grid is also available and can be selected if the user wishes to only display cellular automata processing data, as will be explained later.

The second line Hex Grid Extents Displayed enables the user to define what part of the hexagonal grid is to be displayed if the user wishes only to view a portion of the grid. The Min X and Min Y values define the x,y origin coordinate value from which the bottom left display is to start from, and the Max X and Max Y values define the x,y values to which the top right corner ends. The user can even define a window of display that exceeds the hexagonal grid limits if they wish. These minimum and maximum values are Cartesian coordinates.

Note that the hexagonal grid cells are all of a radius of one, and the structure is of an orientation such that the Cartesian x axis direction is also a straight line in the hex grid, where as it is not on the Cartesian y axis. This therefore means that the x and y Cartesian coordinates for a uniform hexagonal grid of equal x-y dimensions are not equal and the max Cartesian y axis value for the hexagonal grid will always be less than that of the x axis Cartesian coordinate for such a hexagonal grid.

2.3.2 Hexagonal grid display gradient

The hexagonal grid values are displayed with a colour gradient according to the defined colour gradient in the bottom section of the hexagonal grid editor window widget. The user enters a range for the values to be displayed using the defined widget displayed. This gradient widget can be modified to add/delete gradient colour markers by left mouse clicking on the bottom edge of the gradient widget. By selecting a gradient marker, the user can slide and change the position of the marker. Also by selecting a gradient marker, a series of widgets will appear that allow the user to change the colour of the gradient at the location of the marker within the gradient widget. A delete button will also appear enabling the user to delete the current selected gradient marker. To add a marker, the user moves the mouse cursor to just below the bottom gradient widget border, and a ghost marker will appear. The user just needs to position the cursor to where they want to add a new marker and press the left mouse button. To display the hex grid value at any position in the gradient widget, the user needs to move the mouse cursor to just above the top gradient widget border, and by doing this a ghost marker will appear and all the details of the hex grid value and color data at that gradient position is displayed above the gradient widget.

If the user wants to use the edited gradient and have the gradient colours applied to the current displayed hexagonal grid, the user needs to activate the button widget with the label “Update Hex grid colours”. This update is not applied as the gradient is changed simply because of

Once a gradient is created or modified and the user wants to preserve the hexagonal grid display gradient

for future use, the user can save the current gradient to a gradient file by activating the button widget with the label "Save Gradient". A file dialogue will appear and the user can define a name to save the gradient data to. It is recommended to use the existing Gradients sub directory that exists within the directory that the application executable exists within. To import and use an existing colour gradient, the user can activate the button widget with the label "Load Gradient". A file dialogue will appear and the user can define a name to import the gradient data from. The user selects the gradient file and it should be imported and replace the existing colour gradient. If the hexagonal grid display is not updated to the imported gradient colour scheme, the user can do so by activating the button widget with the label "Update Hex grid colours".

At the bottom of this window widget is a checkbox with the label Edit Hex Grid. Selecting this option will display the hex grid editor window widget which is displayed in **Fig 05**.

If the user has the hexagonal grid display active and the mouse is within the hexagonal grid display (**section 2.2**) at the top of this window widget is a display of the current hexagonal grid coordinates of the cursor within that display window widget. If the cursor is hovering over a hexagonal grid cell or voxel, the hex grid index representing that hexagonal grid cell and the value that it contains is displayed below its coordinates. This allows the user to inspect the individual hexagonal grid attributes at a glance for editing or other purposes.

2.3.3 Add edit value to existing grid selection

Below the hexagonal grid attributes display are the hexagonal grid editing tools. First is a checkbox widget with the label "Add to Existing grid". By activating this checkbox, any editing actions performed will add the values of that action to the existing values of the hexagonal grid. Eg if a given hexagonal grid cell has a value of say 3, and an edit action on that hexagonal cell has a value of 2, then the result of that edit action for that hexagonal grid cell is that its value will be 5. Otherwise any edits made will overwrite the existing hexagonal grid value.

2.3.4 Hexagonal grid random number generation

Below this checkbox option is a section to generate a distribution of random numbers to be applied to the hexagonal grid. Because the current implementation of this application has only integer values assigned to the hexagonal grid, most of these selections of random number generation may be incompatible with the current hexagonal grid. However, there are at least two random number generation algorithms that definitely can be used for the integer number data type. Uniform int which is the default random number generator, and bernoulli. To get a better idea how these algorithms generate random numbers and what their inputs and outputs are, a reference to the C++ random library would be a good start, as this application utilises that C++ library for this implementation.

To generate a distribution the user needs to define a random number generator algorithm or method, and a method of selecting a seed for that random number generation. The random number generation method is selected from a drop down list widget next to the label "Generator". With this selection, the user can change the default random number generation seed value by selecting the checkbox with the label "Use Defined Seed Value". Selecting this option, the user is presented with a series of radio buttons to select a user entered seed value, or to use a machine derived and more unknown and random seed value. The user defined and entered seed value is reproducible. That is, the same hexagonal grid random numbers will be generated every time for the selected random number generator. As just hinted, a machine seed value will not be reproducible and is unknown. Two machine derived seed values are given. One based upon the time the random number generation is performed, and another based upon a random selection of seed generators. That is, the computer will select the random number generator and its seed value, not the user.

Once a random number generation method is selected and the seed determination is defined, the user then selects the distribution of those random numbers. As mentioned above, this application currently only accepts hexagonal grid values of integer data type, thus the sure and safe distribution algorithm to use are the default uniform integer and bernoulli methods.

Selecting the uniform integer distribution will display input parameters for this distribution for an upper and lower bounds of the integer numbers to be generated. Use this for generating random integer numbers with a certain range.

Selecting the bernoulli method will display a parameter of percentage of random number distribution having a value of one. The remaining percentage is a value of zero. This is used when the user requires a hexagonal grid or random values being one or zero.

For any random number generation selection, to generate random numbers for the hexagonal grid, the user selects the button widget with the label "Generate Random Hex Grid", and the Hex grid display should change to reflect the random numbers that were generated and assigned to the current hex grid state.

2.3.5 Edit hexagonal grid value

In the section of the Hexagonal grid editor window widget with the label "Edit Hex Automata Grid Values", the user selects an integer value to add to the existing hexagonal grid value if the "Add to existing grid" option is

active (section 2.3.3), or to overwrite the value of a selected hexagonal grid cell if it is not. The value to add to or overwrite a selected hexagonal grid cell value is that given in the input number widget with the label “Value :” If the user moves the mouse cursor into the hexagonal grid window display widget and left clicks on a hovered hexagonal grid cell, the value of that hexagonal grid cell will change according to the user defined value and the activity of adding the edit value to the existing hexagonal grid value or not. This change can also be noticed in the hexagonal grid cell display that is discussed in section 2.3.1.

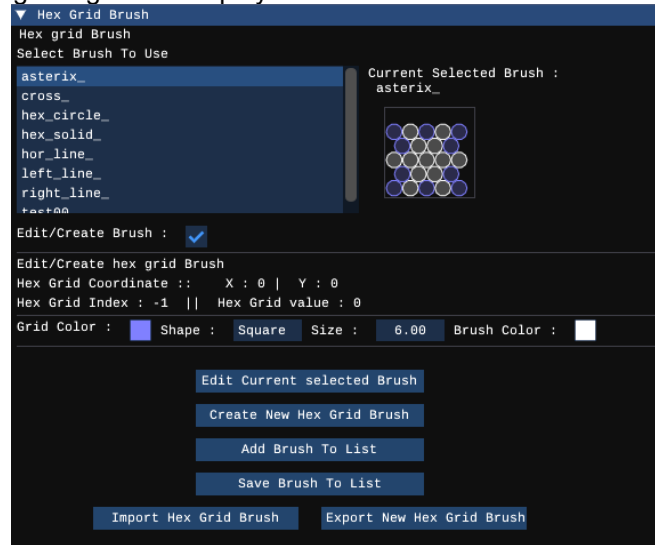


Fig 08

Next to the value entry widget is an option to select a hexagonal grid brush to perform the editing of the hexagonal grid values. By selecting or activating this option an additional window widget will be displayed that should look like that in **Fig 08**. A list of existing hexagonal grid brushes is displayed with a preview of the currently selected brush to the right of the selection list. By selecting a particular brush and then proceeding to edit the hexagonal grid as before, the hexagonal grid cell values will change according to the location and selected brush shape. In the preview the hexagonal brush that will “paint” the hexagonal grid with the designated value is displayed as white circles. The darker blue like colored circles of the brush designate hex grid values that are not effected or changed.

In addition to the default hexagonal grid brushes, the user can edit, create and add their own custom brushes to the list as well as importing an existing brush or saving the current created or edited brush for later use. These brushes are nothing more than a hexagonal grid in themselves that only have two values of one to define the brush to assign an edit value to, and zero not to do so. Thus when creating or editing a brush, the same editing methods are applied and should look the same as editing the main hexagonal grid, but with some restrictions. If the user wishes to add the brush to the list for future use in future projects, then the user should select the button widget “Save Brush to List” The Widget “Add button to list” will add the created/edited brush to the list for use in the current session and will be lost upon opening a new project or restarting the application.

2.3.6 Define a hexagonal grid border value

It may be desirable to define a hexagonal grid value at the edges or border of the created hexagonal grid for certain cellular automata iteration processes. As will later be discussed, if a hexagonal grid process on the edge or border of the hexagonal grid tries to access a hexagonal grid value beyond the boundaries of the hexagonal grid as part of its cellular automata process rules, the value returned is deemed as null and void as designated as a not a number value and is ignored. If it is crucial or unable to define a cellular automata set of rules to deal with such a condition, then a hexagonal grid border value can be defined for all hexagonal grid cells that lie on the edge of the hexagonal grid so as to be able to be used in a cellular automata iteration process. This is the purpose of the section with the label “Hex Grid Border” of the edit hex grid window widget.

To define a hexagonal border with a given value, the user defines a value for all the hexagonal grid cells at the edge of the hexagonal grid, and then clicks on the widget button with the label “Add Hex grid Border”. This action will overwrite whatever value the hexagonal grid edge cells have regardless of what other edit options have been defined. However, these border hexagonal grid cell values can be changed if further edits are performed after this action, so it is advised to only perform this action if a border of a certain value is essential at the end of editing after all other editing has been completed.

2.4 : Creating, Editing and Executing Hexagonal Cellular Automata

For any hexagonal cellular automata process to be performed, there needs to be defined a set of rules that expresses that cellular automata model, and a method for the user to execute and control the automata process of those rules. This is the purpose of the window widget “Hex Surface Automata Rules” (**Fig 09**). It is within this window widget that the user can create, edit, save and import the set of rules that define a cellular automata model and execute and control that model.

In **Fig 09** is an example of an already existing cellular automata model that is ready for execution. An explanation of the process of creating and edit such a model from scratch will be discussed in **Section 3**.



Fig 09

2.4.1 : Automata Rules

The topmost section of this window widget under the label “Automata Rules” is where the individual cellular automata rules are created, edited saved and imported into the current project. Each rule as it is added to the project is assigned a rule number id by the application which cannot be changed, and which is displayed under the label Rule. Each rule however, can be given a name by the user to better describe what that rule applies to in the scheme of the set of cellular automata model. This automata rule name is listed next to the rule id number and under the label “Name”.

Each automata rule is made of and consists of a series of sub rules that is accessed by the user activating the widget button with the label “Edit Rules” that is at the end of the row of each rule.

The Automata Rules section looks to be within a frame or box. If the user were to press the right mouse button within this frame, but not while hovering over any of the existing automata rules, a floating menu with the label “Automata Rules” and an arrow to a sub-menu will appear. By entering this sub-menu, the user will see two selections to add a new rule, or import an existing rule saved to disc.

Selecting “Add Rule”, a new empty rule will appear at the end of the existing rules and be assigned a rule id number by the application.

Selecting “Import Rule”, the user will be prompted to select the file of a saved automata rule in which to import into the project. If a rule is selected to be imported, and upon a successful importation, the imported rule is added to the end of the list of automata rules and assigned a rule id number by the application.

If the user hovers over any of these rules with the mouse cursor and presses the right mouse button, a floating menu with the same label “Automata Rules” and an arrow to a sub-menu will appear. By activating this sub-menu, two additional menu options will appear with the labels “Delete Rule” and “Save Rule”.

Selecting “Delete” will delete the rule that the mouse cursor is hovering over from the list, and the rule

number ids of the remaining rules will be updated.

Selecting "Save Rule", the user will be prompted to nominate a filename, or select an existing file to save the hovered automata rule to. Even though It is up to the user to define the storage directory where to save the rule data to, there is a project directory where the application executable resides called Rules/Rules that is defined for such files.

If a user wishes to delete all of the current rules in this section with one action, just below this frame in which all the project automata rules are displayed is a widget button with the label "Clear All Automata Rules". By activating this button, all the rules of this section will be deleted.

2.4.2 : Automata Rule Results

The next and middle section of the Hex Surface Automata Rules window widget under the label "Automata Rule Results" is where the user defines which rules and in what combination are to be applied and what value is assigned to the hexagonal grid cell if such a rule combination is found to exist or be true for the current state that any particular hex grid cell is in.

Here the user defines a cellular automata model results based upon the set of rules outlined in the section "Automata Rules"

Going from left to right across any rule result row.

2.4.2.1

Active is a flag that sets whether to use or not the defined automata rule result in the cellular automata model iteration process. This is made available so the user can have multiple versions of any cellular automata model that they wish to experiment with, debug or develop without having to save and reload multiple versions of a project.

2.4.2.2

Name is to give the cellular automata rule result a description so the user can better identify what the automata rule result relates or is for.

2.4.2.3

Begin and End relates to the cellular automata iteration start and end iteration steps for that particular automata rule result. This is included to give a flexibility, if needed, to have certain iteration rules and results applied only at certain iteration steps in the process. A kind of temporal or time control of automata iteration processes.

2.4.2.4

Rules is where the user specifies which rules defined in the Automata Rules section are tested or applied to give a certain hexagonal grid cell a nominated result value. The way this currently works is that the user enters in one or more of the rule id numbers corresponding to the rule they wish to use to define a result value to assign to a hexagonal grid cell if the state of that grid cell and its neighbours meet the condition of the nominated rules(s).

To enter just one rule that if the condition(s) of that rule are met by the current state of a hexagonal grid cell and its neighbours the user needs to only enter the rule id number of the rule in this rules entry box. If the user wants to have a combination of more than one rule, each rule id number is separated from each other with the delimiter symbol ":". What this means is that if a rules entry is something like 1:2, then if any hexagonal grid cell has its current iterative state of the automata model that meets the conditions that rule with id number 1 and rule with id number 2 are true or exist, then that hexagonal grid cell is assigned the nominated value as described in **2.4.2.5**.

What this effectively means is something like a programming or logical rule
if(Rule A and Rule B) then
statement.

2.4.2.5

Result is where the user specifies the value that is assigned to a hexagonal grid cell if that hexagonal grid cell and its neighbours meet the conditions of the rule(s) as described in section **2.4.2.4**. Currently only a integer value can be assigned as the hexagonal grid uses only integer values only, but it may be in future this might change with perhaps real or floating point numbers being an option.

2.4.2.6

The Color, Shape, Size and Display widgets all relate to the display of the results in the hexagonal grid cell display (section **2.3.1**). Each automata rule result is displayed as a separate entity within the hexagonal grid cell display. The colour, shape, size can be interactively modified to the users desire to monitor the rule results being processed in each iteration. This can be also useful in the automata development process to aid in making sure each rule result is behaving as expected and to debug the automata model being applied. If the user does not wish to display any or all of these, the user selects the display flag at the end of

the row.

The application processes the rule results as a kind of if (condition) else (condition) like programming statements that has a pseudo code programming statement like

```
if (Rule result A rule(s) met) {  
    hex grid cell = result a  
} else  
    if (Rule result B rule(s) met){  
        hex grid cell = result b  
    } else  
        :  
        if (Rule result N rule(s) met){  
            hex grid cell = result n  
        }  
}
```

What this means is that as soon a a rule results condition is met, the hexagonal grid cell being referenced is given the specified value of that rule result and all others are ignored. The user needs to thus take this into consideration if there are any conflicts of rule results as the first rule that is met and found to be true has the value associated with that result rule assigned to the value of the hexagonal grid cell being referenced.

If a user wishes to delete all of the current rule results in this section with one action, just below this frame in which all the project automata rules are displayed is a widget button with the label “Clear All Results”. By activating this button, all the rule results of this section will be deleted. This will not effect the Automata rules as described in section 2.4.1.

2.4.3 : Save/Import/Clear Hexagonal Grid

The three button widgets with the labels “Import Grid”, “Clear Grid” and “Export Grid” do pretty much as the label description says.

Selecting the “Import Grid” widget button, the user will be prompted to select the file of a saved automata hexagonal grid in which to import into the project. If a grid is selected to be imported, and upon a successful importation, the existing grid will be replaced with the imported grid and all displays and settings relating to the hexagonal updated.

Selecting “Export Grid”, the user will be prompted to nominate a filename, or select an existing file to save the current hexagonal grid data to. Even though It is up to the user to define the storage directory where to save the hexagonal grid data to, there is a project directory where the application executable resides called Grids that is defined for such files.

Selecting clear grid will reset and assign all hexagonal grid cell values to its initial defined value that was nominated upon its creation, and all displays and settings relating to the hexagonal updated.

2.4.3 : Validate Rule Results

Before any cellular automata model iteration process can be performed, the existing cellular automata set of rules and set of automata rule results must first be validated and defined. This needs to occur each step while the user is creating or modifying any of the cellular automata rules and wishes to use those created/edited rules, or after importing a set of cellular automata rules. Any existing cellular automata rules are deleted and the new set validated and defined.

When the user activates this button, the application will go through a process of reading the cellular automata rule definitions and user inputs to test for errors. If any error is found, a dialogue will appear on the screen indicating what caused the error and where it is. The user then needs to rectify the cause and try again to validate the rule results. When/if no error occurs, a dialogue will appear to indicate that the cellular automata rules was successfully defined and the application is ready to perform the cellular iteration process. Any previous rule set will be deleted and replaced.

NOTE: If this process is not performed or the validation failed, no cellular automata process will be performed.

2.4.4 : No Individual Step Constraints option

This is an option for the user to nominate if all of the individual begin and end iteration step constraints are ignored or not. If activated, the application will perform all automata rule results in the iteration process from the lowest defined or begin iteration step up to the largest end iteration step.

Deactivated, each automata rule result will be processed according to their individual begin and end iteration step settings.

2.4.5 : Iteration Process Controls

The four button widgets that look like the control buttons on an audio or video player act in a very similar manner, but act in regards to the iteration process of the application.

2.4.5.1

The first or left most button that has a glyph that looks like a square is a stop button. Activating it will stop the current iteration process and reset the iteration start step to zero. It will not reset the hexagonal grid to any initial state. Thus if the iteration step has reached the end of the maximum number of steps according to the maximum end step of the automata rule results, and the user wishes to continue the iteration with another set of iteration steps, the user needs only to activate this button widget to reset the iteration step back to zero to do so.

2.4.5.2

The second button with the two vertical thick lines glyph that looks like a pause button is exactly that. If the process is automatically performing the iteration steps, the user can activate this button to pause the iteration process. When the user wishes to continue, by pressing this button again, or selecting the third play button, the iteration process will continue.

2.4.5.3

The third button with a glyph that looks like a right pointing arrow acts as a play button. Upon activating this button, the application will from the current iteration step automatically perform each iteration step of each active rule result that is within the permitted minimum and maximum iteration range. Unless the stop or pause widget buttons are activated, the iteration process will continue until the last maximum iteration step is completed.

2.4.5.4

The fourth and last button with a glyph that looks like a right arrow with a vertical line at its tip is the next step button widget. If the play button is not active and the current iteration process step has not reached the maximum permitted step for one or more automata rule result, the user can perform the iteration procedure from the current hexagonal grid automata state to the next manually at their own discretion.

2.4.5 : Step Iteration delay

When the user selects the play button widget as discussed in section **2.4.5.3** the iteration process will proceed automatically until the maximum permitted iteration step is reached. What would occur under normal conditions is that this process may occur very quickly and the user unable to see the individual iteration steps except the end result.

Thus the entry in this widget gives a facility to delay the processing of each step so the user can have some ability to control and view each iteration step in a given time frame. The entry is in seconds with a minimum of 0.1 seconds. If the hexagonal grid is so large and/or the automata rule results so complex or large, this iteration step may become inaccurate and longer if the processing of each step exceeds the nominated delay time period.

2.4.6 : Progress bar

This is a widget that displays the progress of the automata iteration numerically and graphically.

The numerical display of numbers A/B gives the current iteration step A vs the maximum permitted step B.

2.4.3 : Save/Import/ and Clear Automata Rule Set

The three button widgets with the labels “Save Rules”, “Clear All Rules” and “Load Rules” do pretty much as the label description says.

Selecting “Save Rules”, the user will be prompted to nominate a filename, or select an existing file to save the current cellular automata rule results data to. Even though it is up to the user to define the storage directory where to save the automata rule results data to, there is a project directory where the application executable resides called Results/Automata that is defined for such files.

Selecting the “Load Rules” widget button, the user will be prompted to select the file of a saved set of automata rule results in which to import into the project. If a set of automata rule results is selected to be imported, upon a successful importation, the existing set of automata rule results will be replaced with the imported set of automata rule results. To enable the imported set of automata rule results to be applied to the current hexagonal grid, the user will need to validate the set of automata rule results by following the procedure set out in section **2.4.5**.

Selecting Clear All Rules will perform the task of deleting and removing all the Automata rules and Automata Rule Results data and definitions from the current project. This is a clean and efficient method to use if the user wishes to clear the current set of automata rule results and rules to begin creating a new rule set.

3: Create/Edit A Set Of Cellular Automata Rules

The set of cellular automata rules to be used in any cellular automata iteration process are defined in three steps.

1 : Define the cellular automata rule for each state or condition that must be met for that rule to be valid or true.

2 : Aggregate these rule(s) in 1 under a common rule to be used as a single factor to use to determine a value to assign as a single or multiple entity to a automata grid cell.

3: Define the automata rule(s) that must be met to be assigned a value to a hexagonal grid cell.

3 has been discussed in section 2.4.2.

To illustrate the creation of a set of cellular automata rules, the example of creating one for the Conway game of life would be used here as it is the most famous and well known not to mention also not that difficult to implement.

Conways game of life consists of the rules or condition

1. Any live cell with fewer than two live neighbours dies, as if by underpopulation.
2. Any live cell with two or three live neighbours lives on to the next generation.
3. Any live cell with more than three live neighbours dies, as if by overpopulation.
4. Any dead cell with exactly three live neighbours becomes a live cell, as if by reproduction

Leaving these rules unchanged for a hexagonal grid can be tackled in the following manner.

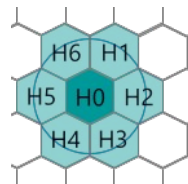


Fig 10

The application defines each hex grid cell that is under consideration for being designated as a live or dead cell as H0, and each of its neighbours in a clockwise order from the top right most hexagonal neighbour being H1, H2, H3, H4, H5, H6. (**Fig 10**)

The application is designed such that the user can reference each of the hexagonal grid cells having a cellular automata iteration procedure performed on them as H0, and each of its neighbours as illustrated in **Fig 10** as H1 H2, H3, H4, H5 and H6. The user can thus define a cellular automata rule as one or more lines of text in the form of a conditional if like statement.

If each grid cell of a hexagonal grid could only have two values to express the state of being alive or dead, such as 0 for a dead cell, and 1 for a live cell, a set of rules can be defined to perform the iteration set of conditions or rules for Conway's game of life as a programmable statement

Thus the rule 1 on Conway's game of life for a live cell to die could be stated as

If $(H0 > 0)$ and $((H1 + H2 + H3 + H4 + H5 + H6) < 2)$
and

rule 2: cell lives on

if $(H0 > 0)$ and $((H1 + H2 + H3 + H4 + H5 + H6) == 2)$ or $((H1 + H2 + H3 + H4 + H5 + H6) == 3)$

rule 3 cell dies

if $(H0 > 0)$ and $((H1 + H2 + H3 + H4 + H5 + H6) > 3)$

rule 4 cell birth

if $(H0 < 1)$ and $((H1 + H2 + H3 + H4 + H5 + H6) == 3)$

3.1 Sub rule statement structure

The Hexagonal Grid automata application iteration process works by the user defining a set of C/C++ like statements similar to the above pseudo code. These rule statements are call sub rules that are assigned to a rule definition that is described in section 2.4.1. When the user activates the "Edit Rules" button widget next to a rule name, a window widget similar to **Fig 11** will be displayed. This is where the user can add, delete, import and save the sub rules that define the cellular automata rule.



Fig 11

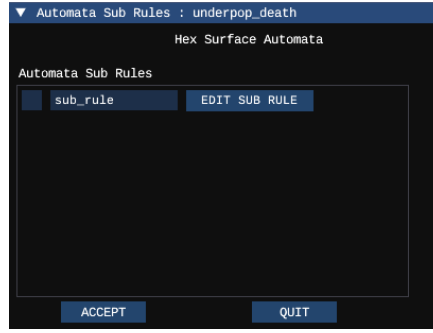


Fig 12

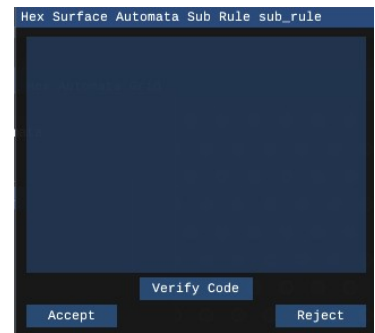


Fig 13

This window widget operates in a very similar fashion as described in section 2.4.1. If the user moves the mouse cursor into the region of what looks like a border frame, and presses the right mouse button, a floating menu with the label “Automata Sub Rules” will appear with an arrow to a sub menu. If the user moves the mouse cursor over this sub menu pointer, a series of menu selections will appear with the selection to add a sub rule or import a sub rule.

If the user selects to create a sub rule, a sub rule is added and the inputs to define the sub rule displayed similar to **Fig 12**. From left to right, the checkbox widget defines the activity of the sub rule. If this check box is active, the defined sub rule will be defined as active and used when the user validates the rule results. The reason for this is so the user can create a number of sub rules and activate and deactivate them as required in development, testing or debugging of their cellular automata model. However, each time a sub rule is activated or deactivated, the user has to validate the rule results as outlined in section 2.4.3.

Next is a name the user can give to the sub rule to give it a description of what its function or purpose is.

Next and at the end is a widget button with the label “EDIT SUB RULE” that when activated opens up the rule definition window widget to enter C/C++ programming like statement(s) that defines the sub rule condition. (**Fig 13**). This window widget is basically a very simple text editor that permits multiple lines to be entered where each line ends with a return to mark an end of a line entry. All cellular automata sub rules can be thought of as a conditional if statement, but without the if word entered. That is, a cellular automata rule condition entered in this editor as a C/C++ like if statement of

will be entered in as

condition A

Thus a sub rule condition statement structure for Conway’s first rule game of life cellular automata model entered in would be something similar to

$$(H0 > 0) \text{ and } ((H1 + H2 + H3 + H4 + H5 + H6) < 2)$$

3.2 Sub rule statement syntax

Every programming language has a particular syntax of statements to establish how the computer is to interpret the text entered by a user into performing a sequence of mathematical computations, logical operations, data retrieval and assignment, so as to return a value that is then used or assigned by the application for a particular task. It is no different here except that the available operations and mathematical functionality is at present very limited but very strict and somewhat different.

The limitation is that the cellular automata rules are effectively a series of logical if statements and thus only return a true or false value to indicate if all the rule conditions have been met or not. The user needs not worry about this return value as it is an internal function of the application.

A limitation of mathematical operations is that only the most basic of mathematical operations are permitted. The current permitted mathematical operations and syntax are listed in **Table 01**

Table 01	
Mathematical Operation	Syntax
Addition	:+
Subtraction	:-
Multiplication	*
Division	/
Modulus	%

Note that the addition and subtraction syntax has a colon : placed before the normal + and – syntax symbols and unless this colon is present before the addition or subtraction symbol, the sub rule condition statement

will not function correctly, or that the condition statement will be invalid and the cellular automata rules will not be accepted.

The sub rule condition statement ultimately must have conditional operators to be able to perform the testing of any cellular automata sub rule. These are very similar to that that exists in most computer code languages and cover all of the most common condition operations These conditional operators are divided into two categories, the conditional operators and the logical operators. The conditional operators perform a comparison test on the mathematical number or result of two numbers to get a true or false false result based upon that comparison, where as a logical condition operator gives a true or false result based upon a logic table.

The current permitted conditional operations are listed in **Table 02**, and the logical conditional operation in **Table 03**

Table 02	
Conditional Operation	Syntax
Not Equal	!=
Less Than	<
Greater Than	>
Less Than or Equal	<=
Greater Than or Equal	>=

Table 03	
Logical Operation	Syntax
EQUAL	==
AND	&&
OR	
NOT	!!

For a cellular automata rule or conditional statement to function and return a true or false value, it must also be able to retrieve the hexagonal grid cell value of the hexagonal grid and all its neighbours so as to perform the cellular automata process of having its current condition tested in relation to its neighbours to return which, if any of the rules of the cellular automata model being conducted are true or not.

In **Fig 10**, and discussed above is a nomenclature for the current hexagonal grid cell (H0) of the cellular automata iteration process to have its conditional rule tested within a cellular automata conditional statement with regards to its neighbours.

And thus as part of the syntax of any hexagonal automata sub rule conditional statement, the nomenclature or token symbol of H0 will retrieve the current hexagonal grid cell value from the hexagonal grid and use it as part of the overall cellular automata rule or conditional statement evaluation. Similarly, the same will occur for each of its neighbours as illustrated in **Fig 10** as H1, H2, H3, H4, H5 and H6.

The final and a very critical part of a correct syntax for any cellular automata conditional rule statement is that every side of every conditional statement is surrounded by the left and right brackets (and). If not, even though the rule may be stated as valid and constructed by the application system, it most likely will not evaluate correctly, or at all. This is especially critical in nested sub rule condition statements. The only exception to this requirement is if a conditional statement like $H0 < H1$ is created as a lone entity. Otherwise any conditional statement similar to $H0 < H1 :+ H2$ must be stated as $H0 < (H1+H2)$

With all this in consideration, the Conway game of life rule 1 to be entered as a cellular automata sub rule with a single conditional statement is

$$(H0 > 0) \&\& ((H1 + H2 + H3 + H4 + H5 + H6) < 2)$$

To Summarise.

The cellular automata sub rule conditional statements only accept the mathematical and conditional operations that are stated in **Table 01** **Table 02** and **Table 03**. and must have all mathematical and conditional statements enclosed in brackets.

With is in mind the remaining Conway game of life rules can be expressed as cellular automata sub rule code

rule 2: cell lives on

$$(H0>0) \&\& (((H1 +H2+H3+H4+H5+H6) == 2) || ((H1+H2+H3+H4+H5+H6) == 3))$$

rule 3 cell dies

$$(H0>0) \&\& ((H1+H2+H3+H4+H5+H6)>3)$$

rule 4 cell birth

$$(H0 < 1) \&\& ((H1+H2+H3+H4+H5+H6) == 3)$$

3.3 Multiple line entries

When the user enters a cellular automata sub rule condition statement into the sub rule editor (**Fig 13**), the user is not confined to enter it in as a single line, but can enter it in as a set of multiple lines. To do so the user simply at the end of a line enters in a carriage return or enter character key to signify an end of line as is done with any text editor. The application will ignore any new line or end of line characters entered. Thus a valid entry for Conway game of life rule 1 entered into the cellular automata sub rule editor is

$$\begin{aligned} &(H0 > 0) \\ &\&\& \\ &((H1 + H2 + H3 + H4 + H5 + H6) < 2) \end{aligned}$$

3.4 Multiple cellular automata sub-rule entries for a single cellular automata rule

An option available when creating a cellular automata rule that requires more than one conditional statement within a sub rule that have a logical and (&&) connecting those conditional statements is to separate the conditional statements into individual sub rule entries. This option exists for rule 1 of Conway's game of life where the $(H0 > 0)$ and $((H1 + H2 + H3 + H4 + H5 + H6) < 2)$ conditional statements have a condition that both must be true for rule 1 of the game of life to be true.

This is because the functionality of the application interprets that each consecutive sub rule of a cellular automata rule must be true for that rule to be true, or otherwise the rule condition is false. A pseudo code for this process can be expressed as

```
if sub-rule A
and if sub-rule B
    :
and if sub-rule N
return true
else
return false
```

What this means is that rule 1 of Conway's game of life can be expressed as two sub-rules sub-rule A, and sub rule B where sub-rule A is

$$(H0 > 0)$$

and sub-rule B is

$$((H1 + H2 + H3 + H4 + H5 + H6) < 2)$$

Here is shown an argument or reason for each sub rule having an activity flag. The user by separating a cellular automata into such constituent parts can by activating or deactivating separate sub-rules experiment with different sub-rules or debug sections of a complete rule to investigate the rule behaviour of a cellular automata model.

What must be specified here is that cellular automata rules that involve or statements (||) or not statements (!!) must exist within a single cellular automata sub-rule. For example rule 2 of Conway's game of life which has the conditional statement

$$(((H1 + H2 + H3 + H4 + H5 + H6) == 2) || ((H1 + H2 + H3 + H4 + H5 + H6) == 3))$$

as part of rule 2 must exist as a single sub-rule entry.

3.5 : Validating the cellular automata sub-rule entry

Once the user is satisfied with the sub-rule entry that they have entered into the sub-rule editor, the user has three options presented by the three button widgets beneath the multiple line entry box that the user can activate. These are

Verify Code :

Activating this button, the user can verify that the cellular automata sub-rule that they have entered or modified is defined as being of valid and correct to be used. This is present so the user can create, in steps or as a whole, and determine if a cellular automata sub-rule is syntactically correct or not and not exit the editor.

If an error in the code entered is detected, a dialogue message will appear indicating what error was found. If no error was detected, a dialogue message will appear indicating that the entered code is valid.

Accept :

Activating this button will perform the same action to determine the validity of the code entered as for **Verify Code**, except that if the code is found to be active, the application will close this widget and assign the code to be used by the application. If an error is detected, it will remain open.

Reject :

Activating this button will close the cellular automata sub-rule editor without validating, saving or assigning the entered code for the application to use. If this editor was opened with existing sub-rule code displayed, any changes made to that code will be discarded upon activating this button widget. Also it best to activate this button if the user wishes to exit this editor if they have opened it just to view or copy text from it into the computers clipboard.

3.6 : Accepting all the cellular automata sub-rule entries

When the user has defined/edited all the cellular automata sub-rules that make up the cellular automata rule that they wish to include as part of the cellular automata iteration model, the user has two options offered to them by the two button widgets located at the bottom of the Automata Sub Rules window widget (**Fig 12**). These are

Accept :

Activating this button will perform the final act of the application to accept all the sub-rules definitions that the user has created/edited to be part of the cellular automata model and close the Automata Sub Rules window widget. No validation of code is performed as this has all been done in the process of creating/editing each cellular automata sub-rule as per sections **3.2** to **3.5**.

Quit :

Activating this button will close the Automata Sub Rules window without the application accepting all the sub-rules definitions that the user has created/edited to be part of the cellular automata model. Therefore this button should only be activated if the user is browsing the existing sub-rules or wishing not to define any changes made for the application to use.

4: Implement A Set Of Cellular Automata Rules

4.1 : Defining the cellular automata model within the Automata rule results

In section **2.4.2 : Automata Rule Results** a description was given on the components of the method to implement and display a cellular automata model upon an existing hexagonal grid which has defined values within that grid that is relevant to the cellular automata model being processed as an iteration. In section **2.5 : Defining and editing the Cellular Automata hexagonal grid to be used**. A description was given on how to create/edit the hexagonal grid upon which any cellular automata model references the data values of each grid cell and its neighbours as part of the cellular automata iteration process. Here an elaboration and expansion is made to clarify what is needed for Conway's cellular automata model to be implemented by this application, and give an insight on how to implement other cellular automata models in this application.

Conway's game of life has only two states in which any grid cell can have that define it as being a live cell or a dead cell. Thus a defined value of one for a grid cell being alive, and a value of zero for a grid cell being dead is a valid and logical choice. This is such for the cellular automata rules expressed as an example for creating a set of cellular automata rules in section 3. Thus the user would need to define a hexagonal grid to have only values of zero or one only. Any other value would render the cellular automata model results giving incorrect results.

Each of Conway's game of life rules are simple and not dependent upon each other which gives a singular result of being alive or dead for each rule which means that each rule exists as a single entity to implement the game of life iteration model. This means that there is one rule entry in the **Automata Rule Results** section of the application for each game of life rule. As described in section **2.4.2.4**, this means that each of Conway's game of life rules is designated as a separate automata rule result that if met modifies the hexagonal grid cell to the value specified in the Result entry widget.

For Conway's game of life rule 1 being death by under population, the user would enter a 0 value in whatever Automata rule result entry corresponding to that automata rule. For rule two to live on, the result value would be 1, rule three death by overpopulation the result value 0, and for rule 4, birth, the result value of 1.

An example cellular automata rules data file exists that the user can load, examine and execute exists in the application executable directory as the pathname

Rules/Automata/conway_game_of_life_HGAR.txt

It is recommended for the user to import this file and reference it to get a better understanding of what has been discussed in this document.

Once these entries for the cellular automata rule results have been defined, or imported, the user can then follow the instructions given in the above sections to validate and define the rules to be used, and create/edit the hexagonal grid to perform the cellular automata iteration model on.

5: Langton's ant

Example of a more complex cellular automata model

Langton's ant cellular automata model states for a normal Cartesian coordinate grid or square lattice that each hex grid cell can be in only one of two states, that of being in and on state or an off state, and that a cell is designated as having an entity identified as an ant that is an autonomous agent pointing in one of four directions along the x or y axis only. To quote for the entry in Wikipedia for Langton's ant

The "ant" moves according to the rules below:

- *At a white square, turn 90° clockwise, flip the color of the square, move forward one unit*
- *At a black square, turn 90° counter-clockwise, flip the color of the square, move forward one unit*

Langton's ant can also be described as a cellular automaton, where the grid is colored black or white and the "ant" square has one of eight different colors assigned to encode the combination of black/white state and the current direction of motion of the ant.

For the hexagonal grid structure that the hexagonal grid automata application utilises, a translation of these rules and data states can be done in the following manner.

A white square is considered as a hexagonal grid cell with a value of 0 to indicate it is in an off state.

A black square is considered as a hexagonal grid cell with a value of 1 to indicate it is in an on state.

The hexagonal grid has three axes and the ant can move in a direction along any of these three axes. That is the ant has six directions of motion.

If the ant moves onto a white or off hexagonal grid cell, the ant direction changes to the left of the direction it was previously pointing in.

If the ant moves onto a black or on hexagonal grid cell, the ant direction changes to the right of the direction it was previously pointing in.

With these definitions a method and set of rules can be devised to emulate Langton's ant on a hexagonal grid structure.

5.1 Deriving a set of states

Unlike for Conway's game of life, a method needs to be developed to define the state of each hexagonal grid cell, and to define the state of the hexagonal grid cell that the "ant" sits on, and the direction that the ant is pointing in. This method must use integer numbers to define these states as that is the data type that the hexagonal grid uses as its basis for cellular automata states and hence defining rule conditions.

The designation of the state of the hexagonal grid being black, on or white off can be defined as a grid cell having a value 1 and 0 respectively.

To define the direction that the ant is pointing to can be defined by which neighbour it is pointing to. There are six neighbours so there will be six directions and thus six numbers to represent which direction the ant is pointing in. Next is a consideration of the state of the hexagonal grid cell that the ant is on that is in combination with the direction that it is pointing in. What this means is that the ant can be pointing in one of six directions on either an on or off hexagonal grid cell, which means that there are twelve possible states that the ant is in.

Add the two states that a cell can have with the number of possible states that a grid cell with an ant in that grid cell, which is twelve, then the total number of possible grid states that exist for cellular automata model of Langton's ant is fourteen.

This will mean that there will be at least a minimum of fourteen automata rule results to assign a value for each of these states to the hexagonal grid.

A method to define the direction and state of the grid cell that the ant sits on can be defined by adding the state of the hexagonal grid cell with a number representing the direction that it is pointing to. Caution needs to be taken into account that these numbers do not coincide or overlap. One method is to use an integer value between 6 and 17, Where the even numbers in this range represents an ant on a zero hexagonal grid and an odd number represents an ant on a one hexagonal grid. Each consecutive even number represents the ant pointing in the direction of its consecutive neighbour from 1 to 6, as does that for the odd numbers. Thus a table of states can be drawn up to represent the state of a hexagonal grid from which to develop the rules, sub-rules and rule results to define the cellular automata model. **Table 4** is such a table

Table 04

Hexagonal Grid State	Representation
0	Hex grid cell off
1	Hex grid cell on
6	Ant on off grid cell pointing in the direction of neighbour 1
8	Ant on off grid cell pointing in the direction of neighbour 2
10	Ant on off grid cell pointing in the direction of neighbour 3
12	Ant on off grid cell pointing in the direction of neighbour 4
14	Ant on off grid cell pointing in the direction of neighbour 5
16	Ant on off grid cell pointing in the direction of neighbour 6
7	Ant on on grid cell pointing in the direction of neighbour 1
9	Ant on on grid cell pointing in the direction of neighbour 2
11	Ant on on grid cell pointing in the direction of neighbour 3
13	Ant on on grid cell pointing in the direction of neighbour 4
15	Ant on on grid cell pointing in the direction of neighbour 5
17	Ant on on grid cell pointing in the direction of neighbour 6

The next part is to define the set of cellular automata rules based on table 4.

Instead of changing the direction of the ant before it moves out of the hex grid cell the direction of the ant is changed as it moves into the hex grid cell. This needs to be done because the cellular automata iteration process works by applying a conditional query on its neighbours as well as its own current state and assigning a value to the hexagonal grid cell based upon that conditional query. A hexagonal grid neighbour cannot have its state changed. What this means is that a hexagonal grid cell can change its hexagonal state on or off after the ant moves out of it in the next iteration based upon **Table 04**. Thus a simple test based upon whether the value of the hexagonal grid cell is even or not will determine the value of the hexagonal grid cell state being on or off in the next iteration step.

The automata rules use table 04 to query if a hexagonal grid cell has one of its neighbours with a state to indicate if an ant is to enter into that hexagonal cell, and from which direction. Thus the current state of that hexagonal grid cell will change and will determine the change of direction of the ant for the next cellular automata iteration. **Table 05** is defined to indicate from which neighbour a given hexagonal cell that has a neighbour with an ant state will move into it or not and change its state to indicate that it has the ant residing within it or not.

Table 05

Hexagonal Neighbour Grid State	Representation
0	Hex grid cell off
1	Hex grid cell on
6	Ant on off grid cell entering from neighbour 4
8	Ant on off grid cell entering from neighbour 5
10	Ant on off grid cell entering from neighbour 6
12	Ant on off grid cell entering from neighbour 1
14	Ant on off grid cell entering from neighbour 2
16	Ant on off grid cell entering from neighbour 3
7	Ant on on grid cell entering from neighbour 4
9	Ant on on grid cell entering from neighbour 5
11	Ant on on grid cell entering from neighbour 6
13	Ant on on grid cell entering from neighbour 1
15	Ant on on grid cell entering from neighbour 2
17	Ant on on grid cell entering from neighbour 3

What can be seen in **Table 05** is that any neighbour harbouring ant entering the hexagonal grid cell from that neighbour has two possible states depending upon whether that neighbour grid cell with an ant is currently on or off.. Upon the ant entering the hexagonal grid cell, that hexagonal grid cell changes its value to indicate a change in direction of the ant based upon its current on/off state and assign a number according to **Table 04** to indicate the direction the ant now is pointing towards. With all this in consideration, the user can now define the cellular automata rules and automata result rules to build and execute Langton's ant automata

model.

The rules to define for Langton's ant are derived from all the possible conditions or states that a single hexagonal grid cell can have in relation to it self, and to its neighbours. This means that the state that a hexagonal grid will be assigned to in a given iteration is based upon its own state if it has a hexagonal grid value greater than 1, and if it has a value zero or one will be assigned to a value in a given iteration based upon its neighbours state.

Therefore, **Table 04** is used to define the conditions in which to determine if a given hexagonal grid cell contains an ant within it, and if not, **Table 05** is used to determine if any other hexagonal grid cell that does not contain an ant within it, but does have a particular neighbour hexagonal grid cell that does, and if the ant in that that neighbour cell is going to enter into it.

Langton's ant has the ant moving every iteration from one hexagonal grid cell into the neighbouring hexagonal grid cell it is pointing at. Langton's rule states that the hexagonal grid cell flips its on/off black/white state as it does so. What this means is that there needs to be a rule or query statement to determine what the current on/off state of any grid cell is, and based upon this rule change the value of the hexagonal grid from on to off, or off to on.

As mentioned above, an even numbered value of a hexagonal grid cell defines a white or off hexagonal grid cell state, where as an odd numbered value of a hexagonal grid cell defines a black or on hexagonal grid cell state. In **Table 04** and **Table 05** this is a common occurrence of hexagonal grid cell state values that is deliberate and designed to have this feature. What this means is that the on/off state of any hexagonal grid cell can be determined by the mathematical modulus operation performed on the number representing its state.

That is in pseudo code the rule to change its state is

if (hexagonal grid cell state $H0 \% 2$) == 0 have off hexagonal grid cell

if (hexagonal grid cell state $H0 \% 2$) != 0 have on hexagonal grid cell

This is all that is needed for determining and implementing Langton's ant rule for when an ant exits a hexagonal grid cell of which all entries in Table 4 is relevant for.

For any hexagonal grid cell that has a state value of zero or one, a query needs to be performed on each of its neighbours to see if any neighbour has an ant within it that is pointing to it and is defined to enter it in the current iteration. This is where **Table 05** is used. As can be seen, an ant entering a hexagonal grid hex from a common neighbouring hex can have a state either of an odd or even number. It does not matter if the neighbouring cell is even or odd, only from which cell the ant enters the hexagonal grid from because this will determine the hexagonal grid cell state that the hexagonal grid cell will be assigned in conjunction with the current on/off state of the hexagonal grid cell.

Example : If a hex grid cell with an empty state value of zero has an ant in its neighbour hex grid cell H5 as depicted in **Fig 10** that is pointing to it and will enter it in the current iteration, Langton's ant rule specifies as defined by this implementation, that the direction of the ant in the next iteration will enter neighbour H1 in the next and thus would be given a value of 6 as defined in **Table 04**.

Conversely, If a hex grid cell with an on state value of one has an ant in its neighbour hex grid cell H5 as depicted in **Fig 10** that is pointing to it and will enter it in the current iteration, Langton's ant rule specifies as defined by this implementation, that the direction of the ant in the next iteration will enter neighbour H3 in the next and thus would be given a value of 11 as defined in **Table 04**.

Therefore there only needs to be six rules to determine from which neighbour hex the ant enters from based upon the neighbours hexagonal grid cell state in **Table 05**.

That is in pseudo code for the rule to determine if a neighbour hexagonal grid cell H5 with an ant is going to enter it can be defined as

if ($H5 == 8$) || ($H5 == 9$) ant entering hexagonal grid H0 from H5

In a similar fashion the rules for all other neighbours with an ant pointing into the hexagonal grid cell to enter it will have a similar structure.

if ($H4 == 6$) || ($H4 == 7$) ant entering hexagonal grid H0 from H4

if ($H6 == 10$) || ($H6 == 11$) ant entering hexagonal grid H0 from H6

if ($H1 == 12$) || ($H1 == 13$) ant entering hexagonal grid H0 from H1

if ($H2 == 14$) || ($H2 == 15$) ant entering hexagonal grid H0 from H2

if ($H3 == 16$) || ($H3 == 17$) ant entering hexagonal grid H0 from H3

Last is a rule for all hexagonal grid cells that do not currently have an ant within that hexagonal grid cell within them is to determine if that hexagonal grid cell is in an on or off state. This is the simplest of all rule conditions in that if the hexagonal grid cell H0 has a value of one, it is on, and if it has a value of zero it is off.

That is

if (H0 == 0) hexagonal grid H0 is off

if (H0 == 1) hexagonal grid H0 is on

From this there are 10 rule conditions from which to use to use to define and assign the automata rule results that define the Langton's ant automata model for each cellular automata iteration process.

Expressed as would be entered as a single line of syntax code for each of these rule conditions for the application to use can have rules defined as in **Table 06**

Table 06

Rule ID	Rule Name	Rule Code
0	White Hex	H0 == 0
1	Black Hex	H0 == 1
2	ant on white hex	(H0 > 1) && ((H0 % 2) == 0)
3	ant on black hex	(H0 > 1) && ((H0 % 2) != 0)
4	move from H4	(H4 == 6) (H4 == 7)
5	move from H5	(H5 == 8) (H5 == 9)
6	move from H6	(H6 == 10) (H6 == 11)
7	move from H1	(H1 == 12) (H1 == 13)
8	move from H2	(H1 == 14) (H2 == 15)
9	move from H3	(H1 == 16) (H3 == 17)

These rules are created within the application as outlined in section 4.

5.2 Deriving a set of Automata Rule Results

The rules defined in **Table 06**, is used to define a set of automata rule results to assign the value of a resultant hex state of the Langton ant model in an iteration process.

These rules only apply to a hexagonal grid in which the ant resides and its neighbours only. Thus no automata rule result needs to be defined for any other hexagonal grid cell. Since the hexagonal grid cell that the ant exists within will flip from its current on/off state to off/on, there needs to be one automata rule for each of these conditions.

For a hexagonal grid cell with an ant in it if it is a white or off cell, assign it to a black or on grid cell. That is

if a hexagonal grid cell has a value that is true for rule id 2, set hexagonal grid cell state to a value of 1

if a hexagonal grid cell has a value that is true for rule id 3, set hexagonal grid cell state to a value of 0

This would mean that in the Automata Rules Results section of the application, the user would create an Automata Rules Result and enter an entry of 2 in the rules entry box, and an entry of 1 in the result entry box for an ant in a white or off hexagonal grid cell, and enter an entry of 3 in the rules entry box, and an entry of 0 in the result entry box for an ant in a black or on hexagonal grid cell.

For a hexagonal grid cell that has a neighbour hexagonal grid cell with an ant that is pointing at and entering the hexagonal grid cell, there are two possible conditions for that ant entering the hexagonal grid cell. That being if the hexagonal grid cell that the ant is entering into is on or off that determines the direction it will face upon entering it. Therefore there needs to be a Automata Rule Result for the ant entering into a hexagonal grid cell with that is in an off or zero state, and another for the ant entering into a hexagonal grid cell with that is in an on or one state.

Taking as an example an ant enter the hexagonal grid cell H0 from its H5 neighbour, if the H0 is in an empty 0 state, the ant is defined to change its direction anticlockwise to point at the hexagonal neighbour H1 in this hexagonal cell H0. This is performed by an Automata Result Rule that has the rule of ID 0 and rule of ID 5 in **Table 06** true, and a value assigned of value 6 as defined in **Table 04**. Conversely, if the hexagonal grid cell is in a on or 1 state, the Automata Result Rule that has the rule of ID 1 and rule of ID 5 in **Table 06** true, will have a value assigned to it of value 11 as defined in **Table 04**.

There are six neighbours to any hexagonal grid cell, so therefore there needs to be 12 Automata Rule Results to cater for each of the possibilities of the ant entering the hexagonal grid cell with one of the two on/off states. This, in conjunction with the two Automata Result Rules for an ant exiting a hexagonal grid cell gives a total of fourteen Automata Result Rules that are required to create a model of Langton's ant in this application, which are listed in Table 07.

Table 07

Automata Rule Result Description	Table 06 Rule(s)	Assigned H0 Result
Ant on white	2	1
Ant on black	3	0
Ant entering white hex from H4	0:4	16
Ant entering black hex from H4	1:4	9
Ant entering white hex from H5	0:5	6
Ant entering black hex from H5	1:5	11
Ant entering white hex from H6	0:6	8
Ant entering black hex from H6	1:6	13
Ant entering white hex from H1	0:7	10
Ant entering black hex from H1	1:7	15
Ant entering white hex from H2	0:8	12
Ant entering black hex from H2	1:8	17
Ant entering white hex from H3	0:9	14
Ant entering black hex from H3	1:9	7

By referencing **Section 2.41**, **2.4.2** and **section 4**, the user can create a cellular automata iteration model for this Hexagonal Grid Automata application.

5.2 Example Langton's ant cellular automata model file.

An example file of the automata model implementing Langton's ant is located in the directory that the application executable resides in under the pathname

Rules/Automata/Langtons_ant_HGAR.txt

The user can import this file into the application and examine and perform the cellular automata iteration process as they desire. When doing so, the user needs to have within the hexagonal grid at least one grid cell that has a value between 6 and 17 to initialise a hexagonal grid cell with an ant pointing in a particular direction, and for the other grid cells not to have a value outside the range of zero or one.

6: Cellular Automata For 2D Cartesian Grid

The discussion in **Sections 4** and **5** on the setting up of the cellular automata model for Conroy's game of life and Langton's ant was for a 2D hexagonal grid system. The Automata Explorer application can also apply a version of these cellular automata rules in its original Cartesian 2D grid and hexagonal close packing (HCP) and Cartesian cubic 3D grid systems.

A regular 2D Cartesian grid is the grid system that most are familiar with as it is the regular 2D coordinate system that is used in fundamental Euclidean mathematics where a coordinate in 2D space is represented as a regular x-y coordinate grid of equally spaced columns (x coordinate) and rows (y coordinate) that when drawn up forms a series of grid cells one unit apart. (**Fig 6.0**) Each of these grid cells defines the cellular automata grid.

Taking any one of these grid cells, there are nine neighbouring grid cells. A representation token for any of Cartesian grid and each of its neighbours can and is represented in this application as in **Fig 6.1**.

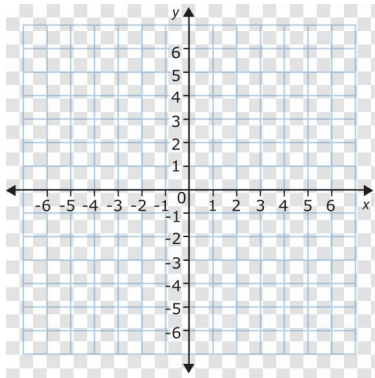


Fig 6.0

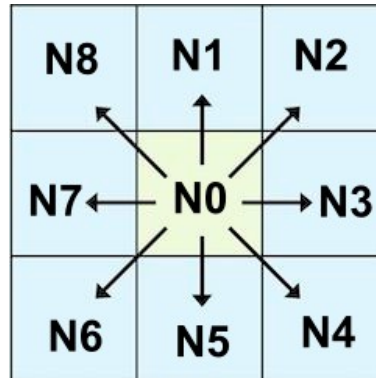


Fig 6.1

As described in **Sections 4** and **5**, a hexagonal grid cell used to define an automata rule is defined as a token in the automata rule as H0 for the hexagonal grid cell under consideration to having its value changed, and H1 to H6 for each of its neighbours as given in **Fig 10**. Here the same is applied where a Cartesian automata grid cell to be changed is defined as N0, and each of its neighbours N1 to N8 as depicted in **Fig 6.1**.

Cellular Automata rules for the Cartesian 2D are defined just as they are for a hexagonal grid, except that the tokens used to defined the central Cartesian grid is N0, and each of its neighbours N1 to N8, where the neighbour under consideration in the rule corresponds to the neighbour as depicted in **Fig 6.1**.

Thus Conroy's game of life model would be

rule 1 live cell to die could be stated as

if $(N0 > 0)$ and $((N1 + N2 + N3 + N4 + N5 + N6 + N7 + N8) < 2)$

rule 2: cell lives on

if $(N0 > 0)$ and $((N1 + N2 + N3 + N4 + N5 + N6 + N7 + N8) == 2)$ or
 $((N1 + N2 + N3 + N4 + N5 + N6 + N7 + N8) == 3)$

rule 3 cell dies

if $(N0 > 0)$ and $((N1 + N2 + N3 + N4 + N5 + N6 + N7 + N8) > 3)$

rule 4 cell birth

if $(N0 < 1)$ and $((N1 + N2 + N3 + N4 + N5 + N6 + N7 + N8) == 3)$

And by using this system, any of the normal Cartesian Cellular Automata rule models that have been created can be defined.

7: Cellular Automata For 3D Cartesian Grid

The discussion in **Section 6** on the setting up of the cellular automata model for a 2D Cartesian grid can also be applied as a 3D cubic version of these cellular automata rules. In this case the central Cartesian grid cell and its neighbours are of a cubic coordinate system and there are 26 neighbours as indicated in **Fig 7.0** and **Fig 7.1**.

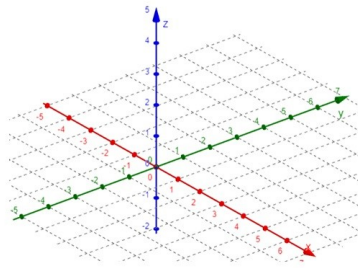


Fig 7.0

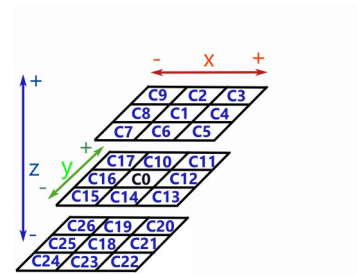


Fig 7.1

Each Cartesian cubic cellular automata grid cell or voxel has an assigned token identifier with the central Cartesian automata grid cell to be changed is defined as C0, and each of its neighbours C1 to C26 as depicted in **Fig 6.1**. The order of each neighbour is from the z axis top to bottom from the central grid cell out to the cell row or y coordinate above the central position, and a rotation in a clockwise direction around this central grid cell. By following the designated token labels for each of the Cartesian cubic grid cells as depicted in **Fig 7.1**, and creating cellular automata rules following the same criteria as given in **sections 4-6** above, a Cartesian cubic 3D cellular automata model can be defined.

Eg. if each Cartesian cell grid can only have a value of zero or one, a rule to change a Cartesian cell grid that has a non zero value of one to zero if all its neighbours above it have a value of one each can be expressed as

$$(C0 > 0) \ \&\& \ (C1 + C2 + C3 + C4 + C5 + C6 + C7 + C8 + C9) == 9$$

7: Cellular Automata For 3D Hexagonal Close Packing (HCP) Grid

The discussion in **Sections 3-5** on the setting up of the cellular automata model for a 2D Cartesian grid can also be applied as a 3D hexagonal close packing (HCP) version of these cellular automata rules. Hexagonal close packing cells or voxels is analogous to a 2D hexagonal grid where the distance from the center of any one hexagon to the center of each and every neighbouring hexagon grid cell is of equal distance. Hexagonal close packing geometry of voxels is in effect, a system of closely packed spheres of equal radius, where each voxel can be represented as a Trapezo-rhombic dodecahedral polygon shape, (**Fig 7.1**)

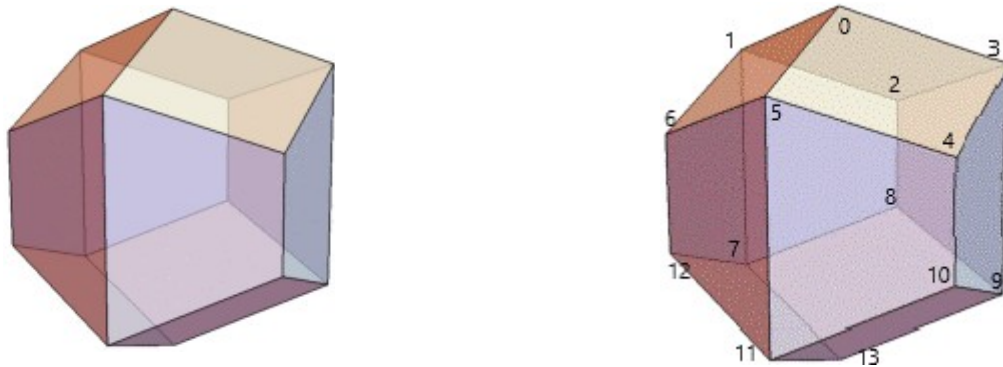


Fig 7.1

The Trapezo-rhombic dodecahedral polygon has twelve faces such that if spheres of equal size in a close packed hexagonal matrix were placed in the same space, the faces of each side would be perpendicular to the points at which each sphere would touch each other. The centre of each face is the location that a voxel sphere would touch its neighbour, and a line through it would connect the centres to each other. Thus the intersections of each of the planes as described above would form the boundaries of each of these polygonal surfaces.

Considering that the Trapezo-rhombic dodecahedral polygon in **Fig 7.1** represents a single HCP cell or voxel, then it can be seen that it has six neighbours in a hexagonal pattern that can be considered as forming the x-y plane as in **Fig 10** of **section 3**. Looking down the z axis, two possible configurations of the neighbouring HCP cells in the layer above the central hcp cell can be possible that are in the form of an equilateral triangle as illustrated in **Fig 7.2a** and **7.2b**.

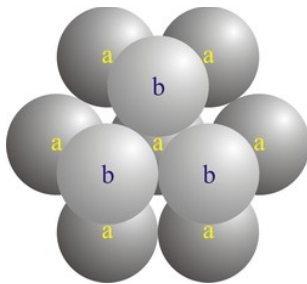


Fig 7.2a

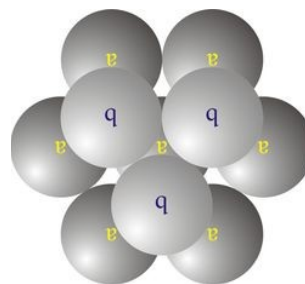


Fig 7.2b

The inverted equilateral triangle arrangement (**Fig 7.2b**) was selected to specify the construction of the faces and conventions to use for the Trapezo-rhombic dodecahedral honeycomb structure for an even HCP layer. This specification requires the three rhombic shaped faces of the top and bottom to be as in **Fig 7.3a**. and **7.3b**. The equilateral triangle arrangement (**Fig 7.2a**) specifies the faces and conventions to use for the Trapezo-rhombic dodecahedral honeycomb structure for an odd HCP layer. This specification requires the three rhombic shaped faces of the top and bottom to be as in **7.3b**.

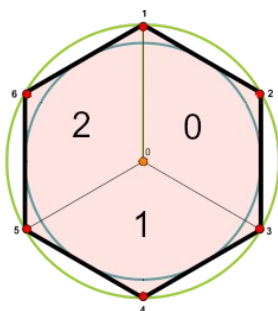


Fig 7.3a

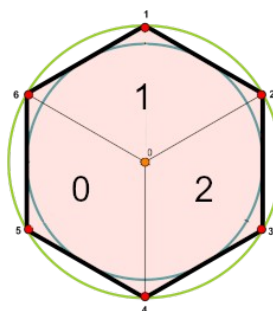


Fig 7.3b

Fig 7.3a corresponds to the neighbours of an even z layer, (ie zero or divisible by 2 or $z \bmod 2 = 0$) and **7.3b** corresponds to the neighbours of an odd z layer, (ie not zero or not divisible by 2 or $z \bmod 2 = 1$)

The number of the face designates an ID to identify each face or neighbour that corresponds to the same change of hexagonal grid coordinates for each of these odd or even z layer neighbours. Eg an odd z layer HCP cell H0 would have an even neighbour cell H1 on face 1 of **Fig 7.3b** in which that even neighbour H1 would have that same odd z layer HCP cell H0 on its face 1 in **Fig 7.3a**.

Looking at Fig 7.1, it can be seen that these faces are consistent for both upper and lower neighbours of any odd or even z layer HCP cell.

What this means is that in formulating the cellular automata rules for a HCP, consideration needs to be taken into account of this difference in orientation or direction of the neighbours of the odd and even z layers of a HCP grid structure.

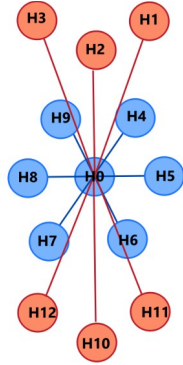


Fig 7.4a HCP even level tokens

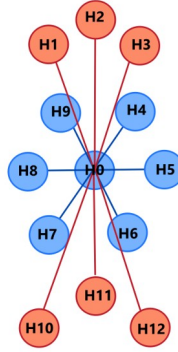


Fig 7.4a HCP odd Level tokens

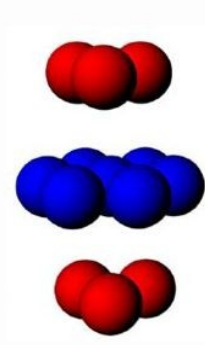


Fig 7.5 HCP even neighbours

Each HCP cellular automata grid cell or voxel has an assigned token identifier with the central HCP automata grid cell to be changed is defined as H0, and each of its neighbours H1 to H12 as depicted in **Fig 7.4a** for an even HCP level, and **7.4b** for an odd HCP level. The order of each neighbour is from the upper to lower z layer with a rotation in a clockwise direction around this central HCP grid cell. By following the designated token labels for each of the HCP grid cells as depicted in **Fig 7.4a** and **Fig 7.4b**, and creating cellular automata rules following the same criteria as given in **sections 4-6** above, a HC 3D cellular automata model can be defined.

Eg. if each HCP cell grid can only have a value of zero or one, a rule to change a HCP cell grid that has a non zero value of one to zero if all its neighbours in the layer above it have a value of one each can be expressed as

$$(H0 > 0) \ \&\& \ (H1 + H2 + H3) == 3$$

8: Application Constraints

The discussion in **Section 5** on the setting up of the cellular automata model for Langton's ant demonstrates the complexity and limitations of the design on this application in creating cellular automata models that have multiple possible states, and especially if an additional attributes such as a direction of movement is concerned. Careful thought and planning needs to be applied when designing such models.

What the Langton's ant model also suggests is that certain cellular automata model classes may not be possible to be implemented in this application as it presently is designed and stands. Langton's ant also is an example of how inefficient an iteration process that model can be in this application since only a few grid cells are relevant for query and change out of what could be a very large grid. Multiple ants would make the implementation more efficient in its use, but then more rules then would be needed to be applied for the ants to interact with each other, and which can escalate on a factorial or exponential scale.

Thus the hexagonal grid automata application as it presently exists is most relevant and applicable to a certain class of cellular automata models that can have their state of existence or attributes represented as a number, and that these cellular automata are not confined to a small part of the entire grid space that they exist in.

When creating a hexagonal or any kind of grid, the user must be aware that the iteration process requires that a copy is made of the existing iteration step which is to be used to create the grid of the next iteration step. Thus the size of the grid may need to be taken into consideration according to the memory resources of the user's computer setup. The size of the hexagonal grid also affects the time it takes to process each step as well as the complexity of the cellular automata model. Currently the application does not utilise any parallel processing, but if the user has a fast enough CPU and system, except for the largest of grids, this should not be of an inconvenience.